

1. (10) Reduction from PARTITION to SUBSET-SUM

Using the fact that PARTITION is NP-hard, show that SUBSET-SUM is also NP-hard. (This is a straightforward problem and we saw the reduction in the opposite direction).

Recall:

Partition: We are given m positive integers $a_1, a_2, \dots, a_m$. The instance is a yes-instance if they can be partitioned into two groups with an equal sum.

Subset-Sum: We are given n positive integers $b_1, b_2, \dots, b_n$, together with a target integer T . The question is if there exists a subset of the integers that add up to exactly T .

solution

the reduction is to ask if there exists a subset sum of

$$T = \frac{\sum_{i=1}^m a_i}{2} \quad (1)$$

on the same set

note if the partition exists, the value each half sums to is exactly this value. deviations means that the sum of each partition does not add up to the total sum of the set which is a contradiction.

this proves that if the mapped to the instance returns false then partition is also false by contrapositive.

if subset sum is true, then one half of the partition is the returned subset. the other half has sum

$$\sum_{i=1}^m a_i - \frac{\sum_{i=1}^m a_i}{2} = \frac{\sum_{i=1}^m a_i}{2} \quad (2)$$

and thus the partition exists

2. (10) Recall that in the **Secretary Problem**, we interview n candidates in random order for one position and wish to hire the best one. The important constraint is that we should make the hiring decision immediately and irrevocably. Suppose we first observe λn secretaries (without hiring) and hire the first candidate thereafter who is better than the best observed in the first phase.

Briefly explain why we hire the best secretary with probability at least $\lambda(1 - \lambda)$. Here, we assume that n is sufficiently large and $\lambda \in (0, 1)$ is a constant.

solution

the actual probability is all combinations of successes, but one combination of success is where the second best candidate appears in the first λ percent of the candidates and the best is in the $1 - \lambda$ percent. the probability of this outcome is $\lambda(1 - \lambda)$ as n goes to infinity, thus the probability is at least this value.

3. (20) In the Max-Coverage problem, we are given a collection of sets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ over a universe U of size n , and an integer k . We would like to recap the analysis of the greedy algorithm (see the pseudocode below) for this problem for a special case.

That is, assume that there exists $Y \subseteq \{1, 2, \dots, m\}$ with $|Y| = k$ such that $|E(Y)| = n$, where $E(Y) := \bigcup_{i \in Y} S_i$. In other words, there exist k sets that cover all the elements.

Algorithm 1: Greedy Maximum Coverage

Input: A collection of sets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$, integer k

Output: A set of indices X

```

1  $X \leftarrow \emptyset;$ 
2 while  $|X| < k$  do
3    $E \leftarrow \bigcup_{j \in X} S_j;$ 
   /* Select the set that covers the maximum number of uncovered
      elements. */
4    $i \leftarrow \arg \max_j |S_j \setminus E|;$ 
5    $X \leftarrow X \cup \{i\};$ 
6 end
7 return  $X;$ 
```

Our goal is to show that $|E(X)| \geq (1 - 1/e)n$, where $E(X) := \bigcup_{i \in X} S_i$, meaning that our solution covers at least a $(1 - 1/e)$ fraction of the whole universe.

- (a) Let X_t denote the first t set indices our greedy algorithm chose. Let $z_t := n - |E(X_t)|$. Explain why $z_{t+1} \leq (1 - 1/k)z_t$.
- (b) Show that $z_k \leq (1 - 1/k)^k n$.
- (c) Show that $z_k \leq \frac{1}{e}n$ using the fact that $1 + x \leq e^x$ for all x .
- (d) Show that $|E(X)| \geq (1 - 1/e)n$.

4. (20) Consider the following two problems.

(Limited) Load Balancing: We have n jobs where each job j has a positive integer size p_j . We have 10 identical parallel machines and a target max load of T . We want to know if it is possible to assign each job to one of the machines so that every machine's load (defined as the total size of jobs assigned to the machine) is at most T .

Partition Problem: We are given m positive integers $a_1, a_2, \dots, a_m$. The instance is a yes-instance if they can be partitioned into two groups with an equal sum.

It is known that Partition is NP-hard. By reducing Partition to Load Balancing, show that Load Balancing is NP-hard.

solution

let

$$S = \sum_{i=1}^m a_i \tag{3}$$

feed the jobs into load balancing with the set A and 8 additional jobs with each one costing $S/2$. the max load is defined to be $T = S/2$.

the solution must make the two “real” jobs be equal to $S/2$ and it is possible iff the partition problem is solvable. the partition in the load balancing is exactly the partition that solves the partition problem.

5. (20) We learned Algorithm “R” in class for drawing a uniform sample from a stream of items. See the following pseudocode.

Algorithm 2: Reservoir Sampling (Algorithm R)

Input: A stream of items $S = x_1, x_2, \dots$; integer $k > 0$

Output: A set of k items (the reservoir)

```

1  $R \leftarrow \{x_1, \dots, x_k\}$ ;
2  $i \leftarrow k + 1$ ;
3 while  $x_i \in S$  do
4   With a probability  $\frac{k}{i}$ ;
5   Evict one item from  $R$  uniformly at random.;
6   Add  $x_i$  to  $R$ .;
7    $i \leftarrow i + 1$ ;
8 return  $R$ ;
```

This algorithm returns a uniform sample R of size k from S , meaning that for each $x_i \in S$, $\Pr[x_i \in R] = k/|S|$. Now suppose each item x_i is associated with a positive integer weight w_i . Modify the algorithm, so that the sample chooses items in proportion to their weight. In other words, your goal is to ensure $\Pr[x_i \in R] = k \cdot w_i / W$ where $W := \sum_{j \in S} w_j$. For simplicity, you can assume that i is sufficiently large so that $kw_j/|S| \leq 1$ for all $j \leq i$ and you only need to modify lines 3-7 in the algorithm.

solution

Algorithm 3: Reservoir Weighted Sampling (Algorithm Weighted R)

Input: A stream of items $S = x_1, x_2, \dots$; integer $k > 0$

Output: A set of k items (the reservoir)

```

1  $R \leftarrow \{x_1, \dots, x_k\}$ ;
2  $i \leftarrow k + 1$ ;
3  $W \leftarrow \sum_{j=1}^k w_j$ ;
4 while  $x_i \in S$  do
5    $W += w_i$ ;
6   With a probability  $\frac{kw_i}{W}$ ;
7   Evict one item  $x_j \in R$  with probability  $\frac{w_j}{kw_i}$ .;
8   Add  $x_i$  to  $R$ .;
9    $i \leftarrow i + 1$ ;
10 return  $R$ ;
```

note, we assume that $\Pr[x_j \in R_{\text{before}}] = \frac{kw_j}{W - w_j}$.

thus after step i we want,

$$\frac{kw_j}{W} = \frac{kw_j}{W - w_j} \cdot \frac{W - w_j}{W} \quad (4)$$

thus we want $\frac{W - w_j}{W} = \text{probability of survival}$.

$$\frac{W - w_j}{W} = 1 - \left(\frac{kw_i}{W} \cdot a \right) \quad (5)$$

where a is the probability of eviction given that an element is chosen to replace an element in R .

$$a = \frac{w_j}{kw_i} \quad (6)$$

i used chat gpt to explain the original algorithm

6. (20) Recall that we can solve maximum cardinality bipartite matching using max flow. Now, consider the following variant: We are given a bipartite graph $G = (L \cup R, E)$. It is convenient to think of L as a set of kids and R as a set of items. An edge (u, v) for $u \in L$ and $v \in R$ means that we can give item v to kid u . The set of items R is partitioned into two groups, A and B . Due to inventory constraints, we can distribute at most c_A items from group A , at most c_B items from group B , and at most c items in total. Our goal is to determine if it is possible for every kid to receive exactly one item.

Solve this problem. Make your solution concise by focusing on how you can modify the solution for the original maximum cardinality matching.

solution

take the original solution to max flow. instead of connecting the start node to each A,B, add two intermediary nodes. make the flow from hte start to the A intermediary node capacity c_a and B intermediary node to be c_b . then connect the intermediary A node to the elements in A and intermediary B to B. make sure the connections out of each item in R is 1 (to the terminal node)

if the max flow is $|A| + |B|$, then it is possible for each child to receive one item.